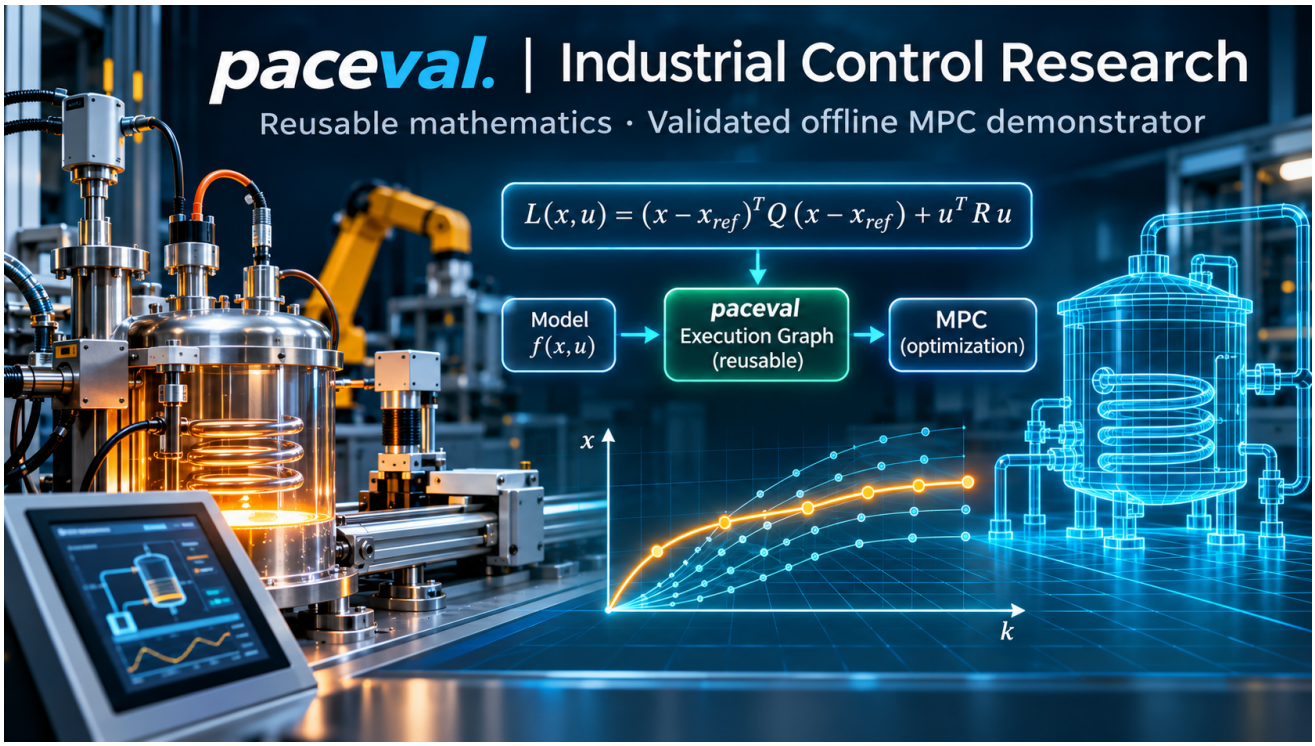
Reusable Mathematical Execution for Industrial Control

An independently validated offline MPC feasibility demonstrator in C++ using the official paceval SDK



Concept illustration only. Industrial installation, plant behavior and hardware device integration are not measured or validated by this project.

Research result at a glance

The original proposal asked whether a reusable mathematical engine could evaluate selected cost and constraint functions within a model-predictive-control workflow. This report presents an implemented, executable feasibility study: eight persistent paceval graphs evaluate a synthetic nonlinear plant transition, stage costs, explicit operating envelopes and an analytic sensitivity inside a C++ finite-control-set MPC loop. Two additional TINC-enabled graphs support a separate numerical audit.

Evidence	Observed with the original library
Four synthetic scenarios	36 offline control steps per scenario; 144 applied actions
Direct mathematical checks	273,760 comparisons against independent native C++; no tolerance failures
Independent precision review	1,152 saved results compared at 110 decimal digits; no tolerance failures
Pointwise interval review	24/24 tested TINC intervals enclose their independently computed value

Scope: a reproducible synthetic software experiment, not a control-system certification or a claim of improved runtime. No numerical speed or hardware speedup figures are presented.

1. Research context and precise boundary

Industrial predictive controllers repeatedly evaluate known objective terms, nonlinear process models and inequality constraints across candidate control sequences. A mathematical-expression engine can provide these isolated functions while a conventional host retains data acquisition, nonlinear programming, selection policies, integration and any plant safety responsibilities. This is the integration boundary proposed in the supplied working paper dated 4 November 2026.

The acados ecosystem already provides optimal-control solvers and derivative-enabled modeling workflows, while CasADi provides symbolic differentiation, code generation and multiple shooting. This experiment does not claim to replace either system or to have implemented their integrations. It examines a selected mathematical component using the official paceval C/C++ interface and a deliberately transparent host-owned discrete candidate search.

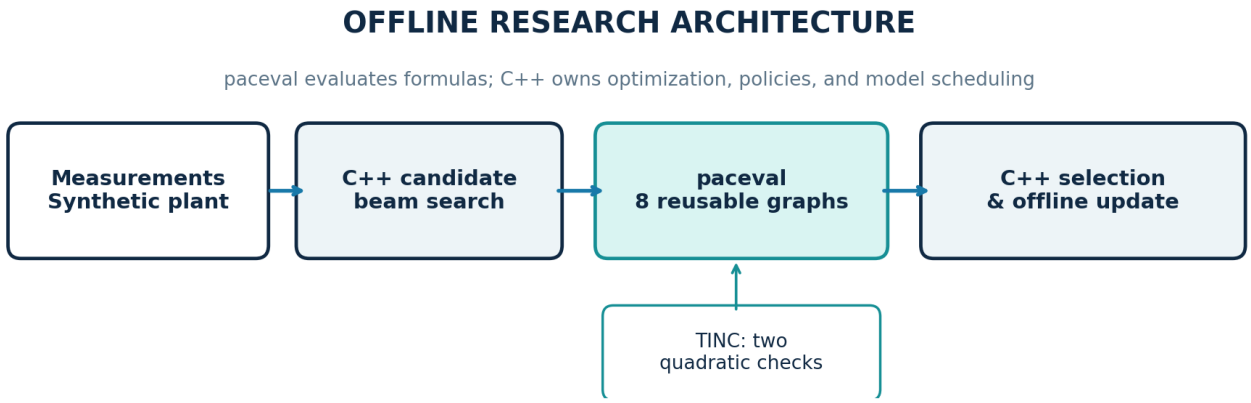


Figure 1. Implemented processing boundary. paceval evaluates mathematical graphs; the C++ host owns candidate generation, constraints as policy, selection and the offline closed loop.

What is demonstrated and what is not

Demonstrated	Outside this experiment
Persistent reusable expression graphs	Real-time bounded latency or scheduler integration
Batch evaluation of multiple candidate predictions	An acados or CasADi plug-in
Discrete action/constraint policies in an offline host	Safety-certified industrial control or hardware-in-the-loop
Independent numerical and finite-action-oracle tests	A physically calibrated industrial plant

2. Synthetic mathematical model

The study uses dimensionless variables and no device data. For discrete simulation index k, the process model is $x(k+1) = a \cdot x(k) + b \cdot u(k) + d(k) + c \cdot x(k)^3$. Here x is the measured scalar state, u is a finite-grid candidate action, d is a synthetic additive disturbance and a, b, c are scenario coefficients. Each control iteration assumes that its currently measured d is constant across that short prediction horizon.

The tracking objective over H predicted steps sums the nonnegative stage costs $q \cdot (x(k+1)\text{-target})^2 + ru \cdot u(k)^2 + slew \cdot (u(k)-u(k-1))^2$. The host admits candidates only when the three squared constraints are nonnegative: $xlim^2 - x(k+1)^2$, $umax^2 - u(k)^2$, and $dumax^2 - (u(k)-u(k-1))^2$. These squared margins are mathematically equivalent to the corresponding absolute-value limits for nonnegative bounds.

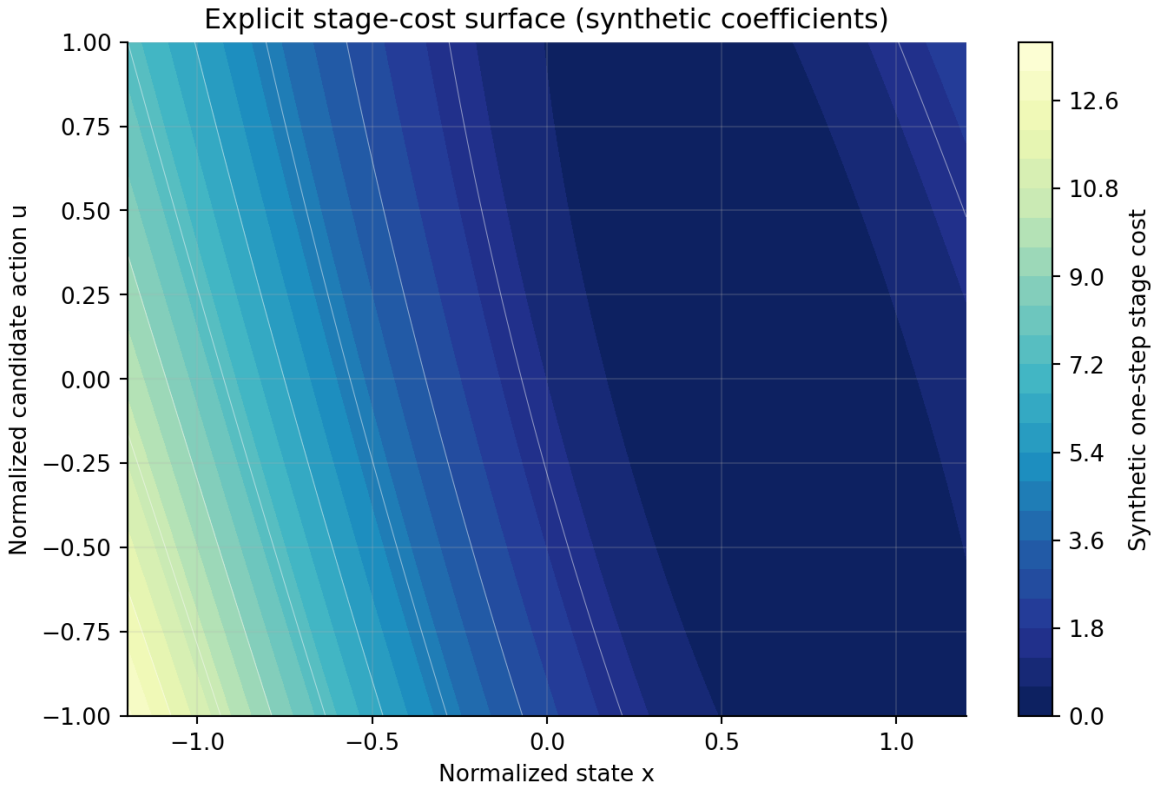


Figure 2. Explicit synthetic one-step stage-cost surface. It illustrates formula evaluation, not measured industrial performance.

Graph inventory used by the original runtime

Graph	Meaning	Mathematical expression
1	Next state	a*x+b*u+d+c*x^3
2	Tracking cost	q*(a*x+b*u+d+c*x^3-target)^2
3 / 4	Effort and slew	ru*u^2; slew*(u-uprev)^2
5 / 6 / 7	Three margins	State, input and rate squared margins
8	Transition sensitivity	a+3*c*x^2

3. Direct paceval implementation

The program includes src/paceval_main.h and compiles the accompanying src/paceval_main.cpp as an application source file. CMake links the separately supplied original Linux shared mathematical runtime. The API source forwards calls into that library: it is not an independently reimplemented mathematical engine. No REST, MCP, dlopen or hand-written ABI layer is used.

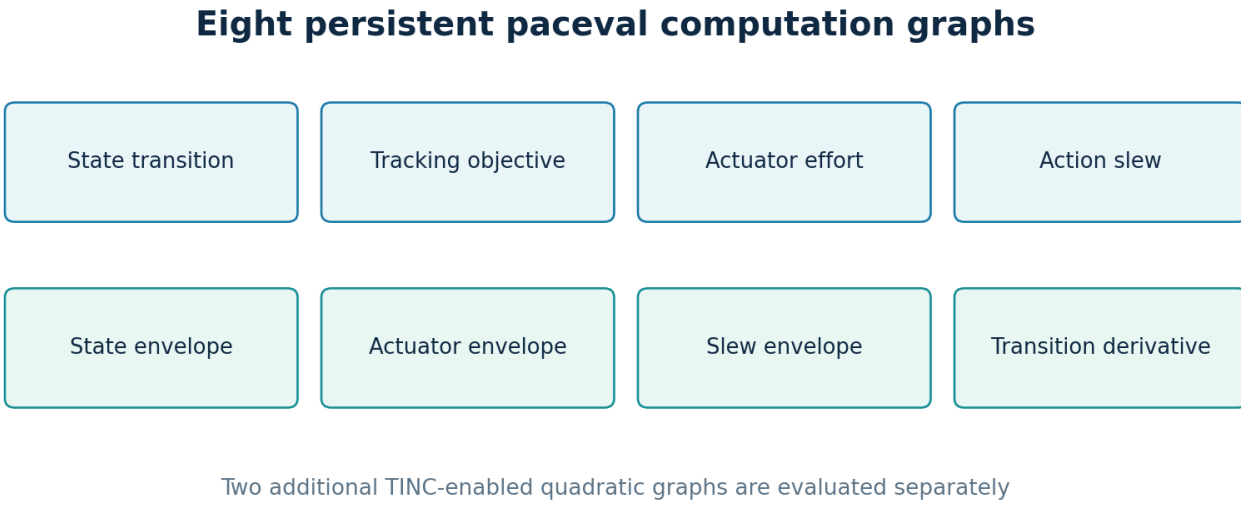


Figure 3. Eight ordinary graph objects are prepared once and reused. Two separate TINC-enabled quadratic graph objects are also instantiated.

Executed API contract

Call	Purpose
paceval_InitializeLibrary	Initialize the original runtime before creating objects
paceval_CreateComputation	Build eight ordinary and two interval-enabled persistent graph handles
paceval_dGetMultipleComputationsResultsExt	Evaluate the eight ordinary graphs across a sample-major input batch; read graph-major output
paceval_dGetComputationResult	Evaluate the two selected TINC graphs pointwise
paceval_GetIsError / paceval_GetErrorInformation	Check graph and runtime status; inspect failures
paceval_DeleteComputation / paceval_FreeLibrary	Release handles before shutting the runtime down

All eight ordinary expressions share the same ordered 14-variable signature. The C++ RAII object owns computation handles, checks each returned error status and releases resources on both normal and exceptional control paths. Graph evaluation is kept separate from host-owned model validation, CSV handling, candidate selection and constraint policy.

4. Offline MPC method and mathematical checks

At each measured synthetic state, the C++ host expands candidates from the action grid {-1, -0.5, 0, 0.5, 1}. The demonstrated rate limit is $|u(k)-u(k-1)| \leq 0.5$. For each prediction depth, candidate input vectors are evaluated through the same eight paceval graphs. Candidates that violate synthetic operating margins are discarded. A beam of the best ranked partial sequences is retained, and the first action from the chosen sequence is applied only to the simulated plant. The procedure then repeats with a new synthetic disturbance.

The beam search is a convenient finite-set research optimizer, not a global nonlinear programming method. To independently check the optimization logic, the study also compares a three-step unpruned paceval-driven search with a separately implemented exhaustive native C++ search. All four preset scenarios reach the same minimum objective in this limited oracle test. With pruning, such an optimality guarantee does not apply.

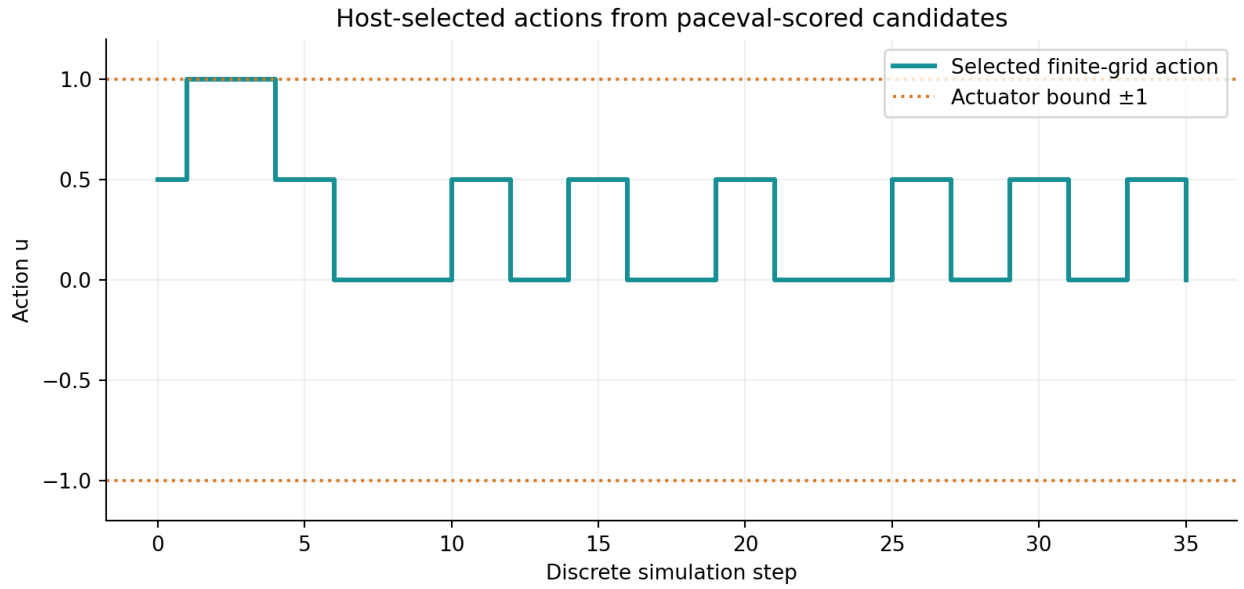


Figure 4. Selected normalized control actions during the first synthetic scenario. There is no physical actuator interface.

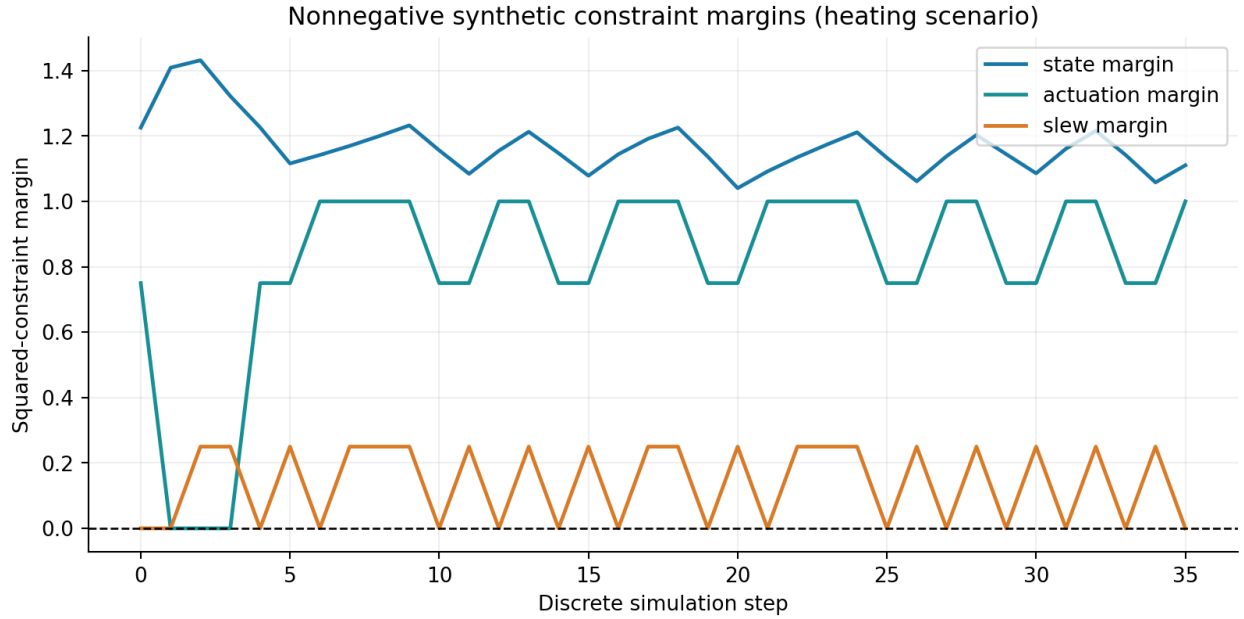


Figure 5. Output margins for the same scenario. Nonnegative values demonstrate compliance with the defined synthetic constraints for the applied actions.

5. Reproducible numerical validation

The evidence is produced with the original supplied paceval Linux runtime (SHA-256: acf63a87df0ee822b467edbca887393d31ec0ea03562e1abf802d2512a5bfb12), the official SDK source pair and a documented synthetic CSV file. Values are checked separately against a native C++ implementation and a Python mpmath implementation at 110 decimal digits; the reference computations do not call paceval.

Test	Observed result
Batch mathematical graph evaluations	273,760 native-C++ comparisons; 0 out-of-tolerance differences
Saved complete control trajectories	4 × 36 = 144 applied actions; 1,152 independently checked graph results
Constraints on applied actions	0 violations of the three documented normalized margins
Independent high-precision arithmetic	Largest absolute reference difference approximately 5.20 × 10^-16
Finite-horizon exhaustive oracle	4/4 cases match for the unpruned three-step search
Compilation and automated tests	GCC 14.2 and Clang 17; 3/3 tests passed with both compilers
Instrumented runtime checks	ASan/UBSan tests passed; external library leak detection not asserted

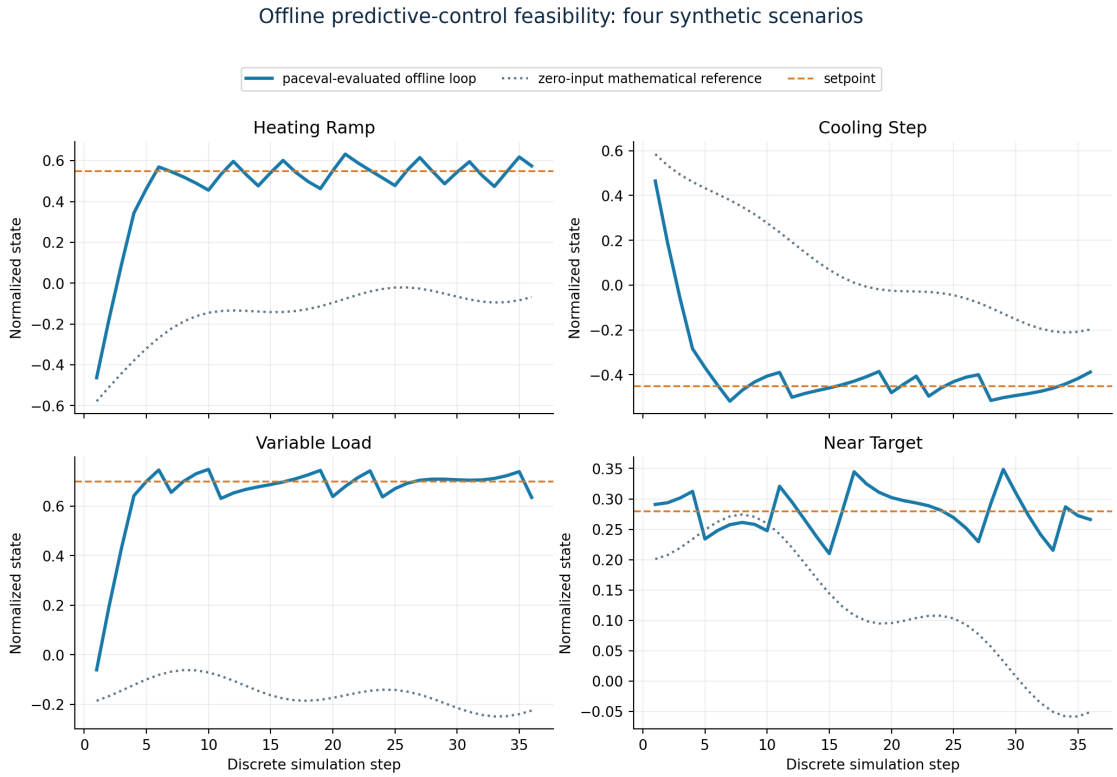


Figure 6. Published offline closed-loop results for all four synthetic scenarios. Dotted lines represent zero-control mathematical reference trajectories, not external benchmarks or industrial measurements.

6. Numerical intervals and limits of the conclusion

The program separately prepares two TINC-enabled graphs for $ru*u^2$ and $q*x^2$ and samples 24 inputs taken from actual offline simulation steps. The CSV stores the point result and lower/upper endpoints reported by the original library. A separate Python script reconstructs the exact binary64 input values from the 17-digit roundtrip CSV and performs 110-digit arithmetic for a strict pointwise inclusion check.

Observed result: 24/24 of these specific high-precision reference values fall within the corresponding reported interval. The result applies only to the selected quadratic expressions and the tested samples. It does not certify general TINC enclosure guarantees, higher-level MPC robustness or a sensor uncertainty model.

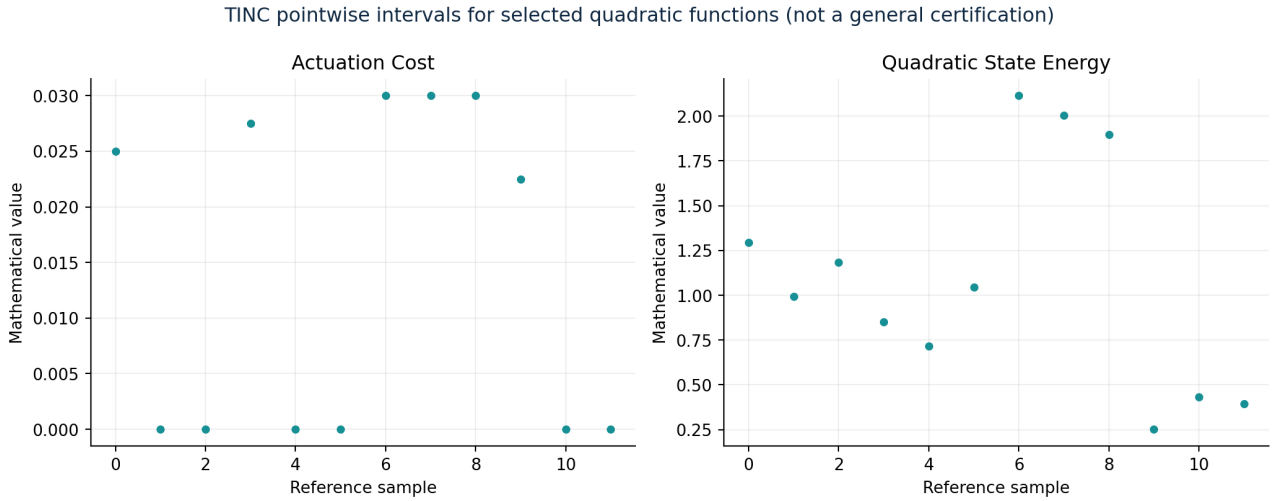


Figure 7. Pointwise intervals for two quadratic functions. At this plotted resolution the tight intervals may appear as markers; the underlying endpoint values and the independent audit are available in examples/tinc_samples.csv.

Three different claims must remain separate

Numerical correctness concerns the implementation of explicitly stated formulas for particular inputs. Optimization quality concerns the choice of candidate actions and its pruning assumptions. Control-system qualification would require validated plant dynamics, scheduling, faults and independent safety analysis. Success in the first does not establish either of the latter two.

7. Reproduction and next research steps

The compact repository contains only the source files, examples, figures, README and this whitepaper. The original runtime is external and is not redistributed. From the repository root, set the absolute path in environment variable PACEVAL_LIB, configure CMake with cmake -S src -B build, build with cmake --build build -j and execute ctest --test-dir build --output-on-failure. The README gives the complete command for reproducing the 36-step study and the independent Python audit.

Included artifact	Purpose
README.md	Project overview, figures, build and validation instructions
src/industrial_control.cpp; src/CMakeLists.txt	Direct original paceval SDK integration and tested host optimizer
src/paceval_main.h; src/paceval_main.cpp	Official supplied 4.25 API source pair with retained AGPL notices
src/validate_outputs.py; src/make_figures.py	Independent high-precision verification and scientific illustrations
examples/scenarios.csv; examples/closed_loop.csv; examples/tinc_samples.csv	Complete reproducible synthetic input, trajectories and interval audit data
figures/*.png	Illustration and output graphs without speed-comparison claims

Responsible extensions

A research partner could next supply a physically identified plant and industrial constraints; introduce an audited cost-function adapter for acados or CasADi; compare numerical derivatives and behavior under uncertain measurements; evaluate failure cases and warm-start behavior; and only then study worst-case timing, multicore architectures and FPGA/ASIC implementation. None of those future engineering gains are asserted as measured outcomes here.

Scientific and technical context

Source proposal: Programmable Mathematics for Industrial Real-Time Control, paceval UG, 4 November 2026. Interface documentation: paceval C/C++ API (SDK 4.25). Related established research tooling: acados documentation and CasADi documentation. These frameworks are cited as context and are not dependencies or tested adapters of the present C++ executable.

Project files: README.md · C++ source · reference audit · published trajectories.

paceval UG · Munich, Germany · https://paceval.com · info@paceval.com